

Informe técnico de avance

Entregable 2

Functional Model of the Authentication and Threat System

System Architecture, Authentication Lifecycle and Threat Modeling

Proyecto:

Componentes criptográficos de autenticación para detección y mitigación inteligente de amenazas en arquitecturas descentralizadas

Becario: David Alexis Córdova Morales

CVU: 756104

Asesora: Gina Gallegos-García

Institución receptora: Centro de Investigación en Computación, IPN

Periodo reportado: 01/09/2025–31/07/2026

Fecha: Julio de 2026

Resumen

El presente entregable documenta la especificación funcional del sistema de autenticación y del modelo de amenazas que servirán como base para el desarrollo del simulador de eventos de autenticación contemplado en el proyecto. A partir de los resultados obtenidos durante la revisión del estado del arte, se definieron las entidades, flujos, amenazas, variables observables y objetivos de detección necesarios para representar de manera consistente el comportamiento de dispositivos IoT dentro de una arquitectura descentralizada.

La propuesta adopta una arquitectura compuesta por cuatro entidades principales: dispositivos IoT, una pasarela de acceso (Gateway), un servidor de autorización y autenticación (Authorization Server) y un broker MQTT. Estas entidades se organizan siguiendo una arquitectura de referencia dispositivo–borde–nube, donde la pasarela actúa como punto central de observación, control y aplicación de políticas de seguridad. El modelo contempla explícitamente las fases de registro, autenticación y renovación de credenciales, así como las dependencias temporales y de estado asociadas a cada una de ellas.

Asimismo, se definió un modelo de amenazas centrado en anomalías relacionadas con el proceso de autenticación. Entre los escenarios considerados se incluyen ataques de repetición (replay), reutilización de nonces, suplantación de identidad, uso indebido de credenciales, accesos sin autenticación previa y comportamientos anómalos asociados a la renovación de credenciales. Para cada amenaza se identificaron las condiciones de ocurrencia, los indicadores observables y los elementos de contexto necesarios para su detección.

El documento también establece el conjunto de eventos, variables y estados que deberán ser generados por el simulador. Estas observaciones incluyen atributos temporales, identificadores de dispositivos, información de sesión, credenciales, tokens, resultados de autenticación y variables de comportamiento que permiten preservar la coherencia entre entidades, estados y transiciones. Con ello se busca generar trazas sintéticas capaces de representar tanto comportamientos legítimos como escenarios de ataque de manera reproducible y controlada.

Finalmente, se definieron los objetivos de detección que orientarán las etapas posteriores del proyecto, distinguiendo mecanismos basados en reglas de aquellos basados en comportamiento. La especificación funcional obtenida constituye la referencia de diseño para el desarrollo del simulador, la generación de conjuntos de datos sintéticos y la futura evaluación de mecanismos inteligentes de detección de anomalías en procesos de autenticación IoT.

Contents

Resumen Ejecutivo	i
1 Introduction and Motivation	1
1.1 Context of the Project	1
1.2 Objective of the Simulator	2
1.3 Scope of the First Version	2
1.4 System Assumptions	3
2 Architecture	5
2.1 Device	5
2.2 Edge Gateway	7
2.3 Authorization Server	8
2.4 MQTT Broker	9
3 Flows	11
3.1 Registration Flow	11
3.2 Authentication Flow	12
3.3 Renewal Flow	14
3.4 Attack Flow: Replay	15
4 Threat Model	16
4.1 Replay	16
4.2 Nonce Reuse and Timestamp Anomalies	17
4.3 Impersonation	18
4.4 Flooding and Protocol Abuse	18
4.5 Access Without Authentication	19
4.6 Abnormal Renewal	19
4.7 Attack Summary	20
5 Observable Variables	21
5.1 Session and Identity Fields	21
5.2 State-Machine Transition Fields	21

5.3	Token and Nonce Fields	21
5.4	Network and MQTT Fields	22
5.5	Temporal and Session-Aggregate Fields	22
5.6	Attack-Variant Signal Fields	22
5.7	Ground-Truth Labels	22
5.8	Summary Table	23
5.9	Dataset Structure	23
6	Detection Targets	25
6.1	Detection Objectives	25
6.2	Attack-to-Signal Mapping	26
6.3	Label Leakage and Honest Evaluation	26
6.4	Expected Detection Difficulty	27
6.5	Non-Objectives	27
7	Authentication State Machine	28
7.1	States	28
7.2	Valid Transitions	29
7.3	Invalid Transitions (Anomaly Injection)	32
A	Dataset Schema	34

Introduction and Motivation

1.1 Context of the Project

The Internet of Things is now pervasive across smart homes, healthcare, transportation, industry, and connected infrastructure. In all of these settings, devices continuously exchange data with gateways, cloud platforms, and applications, which makes authentication a fundamental security mechanism: a device must be correctly identified and verified before it is allowed to communicate or to access a resource. Securing that authentication is difficult. IoT devices are typically constrained in computing power, memory, energy, and bandwidth, and they are deployed in heterogeneous, sometimes physically exposed environments that widen their attack surface. The widely used MQTT messaging protocol illustrates the tension well: it is efficient and suitable for constrained devices, yet it raises real concerns around device identity, access control, session management, token validation, and re-authentication. These concerns map naturally onto a Zero Trust perspective, where trust is established, limited, and continuously reassessed throughout the communication lifecycle rather than granted once and assumed thereafter.

A central obstacle in IoT security research is the scarcity of realistic, well-structured datasets focused specifically on authentication events. Existing datasets describe general network traffic or broad attack campaigns, but rarely the authentication process itself: they seldom indicate whether a device is legitimate, whether a token is valid, whether access to a topic is authorized, or whether a session contains suspicious authentication behaviour. This missing detail makes it hard to train and evaluate machine-learning models dedicated to authentication anomaly detection. Collecting such data from real deployments is equally problematic, because it may expose sensitive information about devices, users, credentials, or infrastructure, and it undermines the reproducibility of attack scenarios that Zero Trust evaluation requires. A controlled simulator that generates synthetic authentication-flow data, covering both normal behaviour and selected attacks, is the practical route out of this gap. The problem it addresses can be stated as follows.

How can we design a simulator of IoT authentication flows that generates realistic synthetic data for studying and detecting authentication anomalies?

1.2 Objective of the Simulator

The simulator is a programmatic scenario engine whose objective is to produce a labelled synthetic dataset covering the full authentication lifecycle of MQTT-based IoT devices. It reproduces the six lifecycle phases — discovery, pairing, enrollment, authorization, MQTT session establishment, and re-authentication — and injects nine authentication-centric attack variants at controlled rates during those phases. Every simulated event is emitted with a ground-truth label and with the observable variables needed to reconstruct the context in which the event occurred, so that anomaly detectors can be trained and evaluated against a reference of known truth. The simulator is therefore not a testbed or a broker: it is a data-generation tool whose output is the input to a downstream detection pipeline.

Three properties are treated as intrinsic to this objective. *Coverage* means every attack in the threat model (Section 4) can be injected at any point in the lifecycle where it is meaningful. *Reproducibility* means a given scenario configuration and random seed produces a byte-identical dataset, so that detection experiments can be compared and defects can be traced back to the exact events that caused them. *Traceability* means every emitted variable maps back to a specific decision in the functional model, so that the meaning of any field can be checked against the model that produced it.

1.3 Scope of the First Version

The first version of the simulator is deliberately narrow. It targets the authentication lifecycle rather than the full space of IoT security, and it fixes several protocol and topology choices to keep the design tractable and the dataset interpretable. Within this scope:

- **Messaging protocol.** Only MQTT (version 5) is modelled. Alternative IoT protocols (CoAP, AMQP, HTTP) are out of scope.
- **Authentication framework.** OAuth2/ACE with proof-of-possession is modelled at the message-flow level: token issuance, scope binding, PoP challenge, and expiry are all reproduced. In the default generation path the underlying cryptographic operations (TLS handshake, ECDH, signature verification) are represented symbolically rather than computed; the domain model additionally ships a real secp256r1 ECDH / Blake2s implementation that can be exercised end-to-end through an optional flag. Cryptographic cost is reflected in the per-step latency model; bit-level cryptographic protocol execution is not the object of study.
- **Access control.** ABAC is applied to publish/subscribe requests with a small, fixed attribute set (device ID, requested topic pattern, and operation) matched against a per-device policy table, with default-deny for any request that matches no rule. Full policy languages such as XACML are out of scope.

- **Topology.** A pool of edge gateways (default: five) mediates device–cloud interactions; each session is handled by exactly one gateway, and all gateways share a single device pool, apply identical policy, and keep no cross-gateway state. A single authorization server and a single broker complete the cloud side. Federated authorization servers, broker clusters, and richer multi-gateway topologies (per-gateway device pools, inter-gateway coordination) are out of scope.
- **Threat model.** Nine attack variants are injected, all from a single source and all non-invasive (Section 4). Coordinated attacks across multiple devices (Sybil, distributed DoS, bad-mouthing) and invasive attacks that succeed against valid credentials are out of scope.
- **Detection layer.** The simulator produces the dataset; a companion supervised ML pipeline consumes it (Section 6). Unsupervised, semi-supervised, and online-learning detectors are out of scope for the first version.
- **Physical layer.** No physical-layer attacks (jamming, side-channel, hardware tampering) are modelled. Traffic is represented at the MQTT and authentication-message level.

Every item explicitly labelled out of scope corresponds to a natural extension direction rather than a design exclusion.

1.4 System Assumptions

The functional model rests on the following assumptions. They are stated here so that the meaning of any downstream event or detection claim can be interpreted against them.

- **Trusted gateway.** The edge gateway is assumed to be uncompromised. It is the vantage point from which every event is observed and labelled, and its own integrity is not part of the threat model. The gateway-trustworthiness problem is acknowledged in the literature as an open architectural question and is left to future work.
- **Trusted authorization server.** The AS is assumed to be available, correct, and uncompromised. Token issuance, PoP challenge generation, and revocation decisions produced by the AS are treated as ground truth.
- **Trusted broker.** The MQTT broker honours the ACLs the gateway enforces upstream and does not itself act maliciously. Broker-internal compromise is out of scope.
- **Correct cryptographic primitives.** All cryptographic operations (ECDH key agreement, token signing, PoP verification, TLS confidentiality and integrity) are assumed to be mathematically correct. The simulator does not model cryptographic breaks; attacks target the surrounding protocol logic (freshness, binding, authorization scope), not the primitives themselves.

- **Pre-provisioned identity.** Every device carries a persistent pre-shared key installed at manufacturing time (an X.509 certificate or PUF fingerprint would serve the same role but are not modelled in this version). The provisioning process itself is not simulated.
- **Simulated time.** The simulator runs on a logical per-session clock: timestamps advance by sampled per-event delays rather than by real elapsed time. Timestamps are monotonic within a session; clock drift and time-synchronization errors are not modelled except when explicitly injected by the `timestamp_inconsistency` attack.
- **Reliable transport.** The underlying network is assumed to deliver messages without loss, reordering, or arbitrary latency. Delivery timing is modelled parametrically (per-event-type, class-dependent latency distributions), but stochastic packet loss and network partitions are not.
- **Single active session per device.** At any given simulated time, each device holds at most one open MQTT session with the broker. Concurrent sessions from the same device are not modelled.
- **Bounded device population.** The device population is fixed at scenario start; devices do not join or leave the network dynamically during a run. Onboarding events at the start of a scenario are modelled, but mid-scenario churn is not.

Architecture

The simulator reproduces a four-entity architecture: a fleet of IoT **devices**, an **edge gateway** that mediates all their traffic, an **authorization server** that issues and validates tokens, and an **MQTT broker** that carries application-level messaging. This mapping mirrors the three-layer device–edge–cloud reference architecture identified in the literature, with the authorization server and broker both residing in the cloud layer. Figure 2.1 shows the layout.

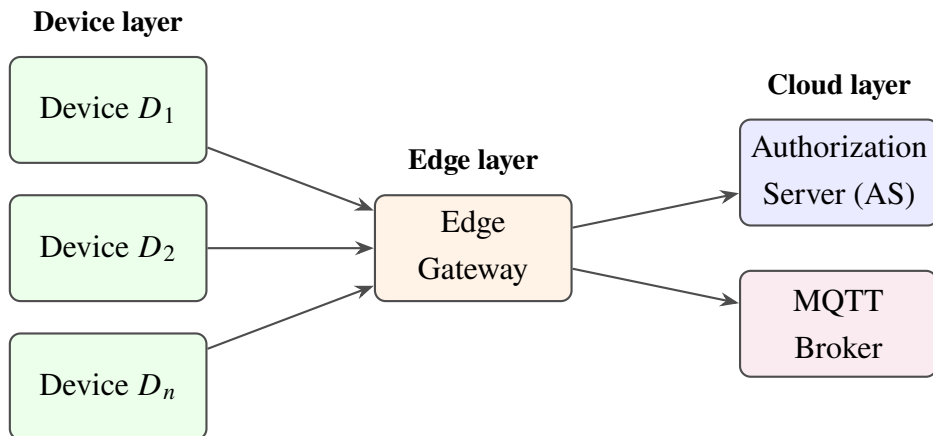


Figure 2.1: Reference architecture of the simulator. Devices communicate only through the gateway, which brokers all interactions with the cloud-side authorization server and MQTT broker.

The four entities are treated as independent state machines that exchange messages. The simulator instantiates each entity, drives their interactions through a scenario script, and logs every exchange. The remainder of this section describes each entity’s responsibilities, lifecycle states, internal attributes, and interfaces.

2.1 Device

A device represents a resource-constrained IoT endpoint — a sensor, an actuator, or an embedded controller. It is the entity being authenticated and the entity whose behaviour the anomaly detector will ultimately judge.

Responsibilities. The device is responsible for initiating its own onboarding, requesting tokens when it needs to access the broker, publishing messages under its authorized topic scope, and re-authenticating when its token expires. It holds a persistent identity and derives all subsequent credentials from it.

Lifecycle states. A device moves through a small set of lifecycle states over its existence in the simulation. Table 2.1 defines each state, the transition that enters it, and what the device is allowed to do while in it. The state is the primary driver of message-emission rules: several attack variants are precisely defined as messages emitted from a state that does not permit them.

State	Entered on	Meaning and permitted actions
unregistered	Scenario start	Device is unknown to the AS. Only discovery and pairing messages are permitted; token requests and MQTT connections are rejected.
pairing	Discovery success	ECDH pairing in progress. A confidential channel is being established but no identity has been verified yet.
enrolled	Enrollment ACK from AS	Device identity is recorded in the AS registry. The device may now request tokens but has no active session.
authenticating	Token request sent	Token request / PoP challenge in progress. The device is neither idle nor active.
active	CONNACK received	Device holds a valid token and an open MQTT session. Publishing and subscribing within scope are permitted.
renewing	Token nearing expiry	Re-authentication in progress on an already-open session. The MQTT session remains open; only the token is being replaced.
blocked	Rate-limit exhaustion or revocation	Device is temporarily prevented from initiating any authentication or session action. Existing session (if any) is closed.

Table 2.1: Device lifecycle states.

Attributes. Each device carries the attribute set summarized in Table 2.2. These attributes are the device's internal state; a subset of them is exposed to the gateway on each message and therefore appears in the observable feature table (Section 5).

Attribute	Type / domain	Description
device_id	UUID	Unique identifier assigned when the device pool is created.
device_type	{sensor, actuator, gateway_client}	Coarse device class, reported in the event log.
psk	32-byte key	Persistent pre-shared key provisioned at manufacture time; only its Blake2s hash ever leaves the device.
ecdh_keypair	secp256r1 key pair	Ephemeral pairing key generated during the pairing step (not persistent).
credential_valid	Boolean	Whether the device presents genuine credential material; flipped by attack modules.
lifecycle_state	See Table 2.1	Current state in the lifecycle state machine.
access_token	Token id + issue time, or null	Token currently held; its bound scope and expiry are kept by the AS.
failed_auth_count	Integer	Running count of recent authentication failures; feeds the gateway's rate-limiter.
trust_score	Float [0, 1]	Continuous Zero-Trust score, degraded on each observed deviation.
battery_level / firmware_version	Float (%) / version string	Reported device telemetry exposed in the feature table.

Table 2.2: Device attributes.

Interfaces. Devices communicate exclusively with the gateway; they never contact the authorization server or the broker directly. This is a deliberate design choice that reflects the edge-first security model: the gateway is the only component that spans both the constrained side and the cloud side, and it is therefore the only component that can enforce policy consistently across both.

2.2 Edge Gateway

The edge gateway is the central enforcement point of the architecture. It proxies every device request, applies policy, and forwards only what the policy permits. It is also the point at which the simulator captures and labels events, because the gateway sees every authentication attempt, every token presentation, and every publish action — which no other single entity does.

Responsibilities. The gateway performs six functions: (i) terminating the TLS channel from the device and re-establishing it toward the cloud, (ii) validating tokens against the authorization

server, (iii) enforcing ABAC policy on publish/subscribe requests, (iv) rate-limiting devices that fail authentication repeatedly, (v) detecting simple protocol-level anomalies such as duplicate nonces or replayed sequences within its own memory window, and (vi) emitting the labelled event records that form the simulator’s output dataset.

Attributes. The gateway is stateless with respect to any single message but maintains the per-session and cross-session structures shown in Table 2.3. Together these structures define both the gateway’s enforcement capability and the *replay window* used throughout the threat model.

Attribute	Type / domain	Description
gateway_id	UUID	Unique identifier of the gateway instance.
session_table	Map session_id → record	Open sessions with their current token, device identity, and last-seen timestamp.
nonce_cache	Bounded LRU with TTL	Recently seen PoP nonces; the TTL defines the replay window.
token_cache	TTL-expiring cache	Recently seen token identifiers, used for local replay detection.
failure_counter	Map device_id → int	Per-device count of recent authentication failures; drives rate-limiting.
policy_table	ABAC ruleset	Mapping of device_id (and attributes) to permitted topic patterns.
event_log	Append-only stream	Labelled records emitted to the dataset output pipeline.

Table 2.3: Edge gateway attributes.

Interfaces. The gateway exposes a device-facing interface (device authentication, device publish/subscribe) and two cloud-facing interfaces (token validation against the authorization server, MQTT forwarding to the broker). All interfaces are message-oriented: the gateway never blocks waiting for a device, so device slowness or silence does not degrade the gateway’s ability to serve others.

2.3 Authorization Server

The authorization server (AS) is the sole issuer and validator of access tokens. It corresponds to the OAuth2/ACE Authorization Server in the retained protocol stack and separates authentication proper from authorization decisions.

Responsibilities. The AS issues short-lived tokens bound to a device identity, a scope (the set of topics the token grants access to), and an expiry timestamp. The AS also revokes tokens on

demand, either because a device has been decommissioned or because the gateway has flagged its behaviour as anomalous.

Attributes. The AS maintains the three tables shown in Table 2.4. It holds no application-level state — it never sees the messages the devices publish.

Attribute	Type / domain	Description
device_registry	Map device_id → PSK hash	Enrolled devices, each stored as the Blake2s hash of its pre-shared key.
active_tokens	Map token_id → record	Currently valid tokens with their bound device_id, scope, issue time, and expiry.
revocation_list	Map token_id → reason	Tokens explicitly revoked before their natural expiry.

Table 2.4: Authorization server attributes.

Interfaces. The AS is contacted by the gateway — for enrollment and token issuance — and by the broker, which revalidates the presented token with the AS on each CONNECT before opening a session. Devices never contact the AS directly. This funnelling is what allows the gateway to serve as the single point of policy enforcement even though the ultimate identity decision is made in the cloud.

2.4 MQTT Broker

The broker is the message-routing entity. It accepts publish and subscribe requests from authenticated devices (via the gateway) and delivers messages to their subscribers. The simulator models an MQTT v5 broker, because v5 natively supports challenge-based authentication and re-authentication without reconnection — both of which are required to model the Zero Trust posture faithfully.

Responsibilities. On each incoming CONNECT the broker revalidates the presented token with the authorization server before opening a session. It then delivers messages at the requested Quality of Service (QoS 0, 1, or 2) and enforces a topic-prefix scope on publishes, rejecting any publish whose topic falls outside the configured prefix. Per-session publish, subscribe, and byte counters are tracked for the feature table.

Attributes. The broker maintains the per-session structures shown in Table 2.5. All state is scoped to a single session; no cross-session structure is kept.

Attribute	Type / domain	Description
sessions	Map device_id → record	Open sessions with clean-session flag, keep-alive, connect/last-seen timestamps, and per-session counters.
delivery_queues	Map session_id → queue	Pending QoS 1 and QoS 2 messages awaiting acknowledgement.
session_counters	Map session_id → counters	Per-session counts of publishes, subscribes, and bytes exchanged; exported to the feature table.
topic_prefix	Prefix string	Configured topic-prefix scope; publishes outside this prefix are rejected.

Table 2.5: MQTT broker attributes.

Interfaces. The broker is contacted only by the gateway. As with the AS, this funnelling is a modelling choice: it lets the simulator concentrate all observation at the gateway without loss of visibility, since every broker-bound message must first pass through the gateway.

Chapter 3

Flows

The simulator drives its entities through three nominal flows and, on top of these, injects attack variants that deviate from the nominal path at specific transitions. This section describes the three nominal flows — **registration**, **authentication**, and **renewal** — and illustrates a representative attack flow (replay) to make the deviation mechanism concrete. The complete attack catalog is enumerated in Section 4.

3.1 Registration Flow

Registration is the one-time process by which a device transitions from **unregistered** to **enrolled**. It combines discovery (the device locates the gateway), pairing (a confidential channel is established via ECDH), and enrollment (the device’s identity is verified and recorded in the AS registry). The distinction between pairing and enrollment is essential: pairing produces a confidential channel; enrollment authenticates the device within that channel. Treating a confidential channel as proof of identity is a recurring design error and is explicitly avoided here.

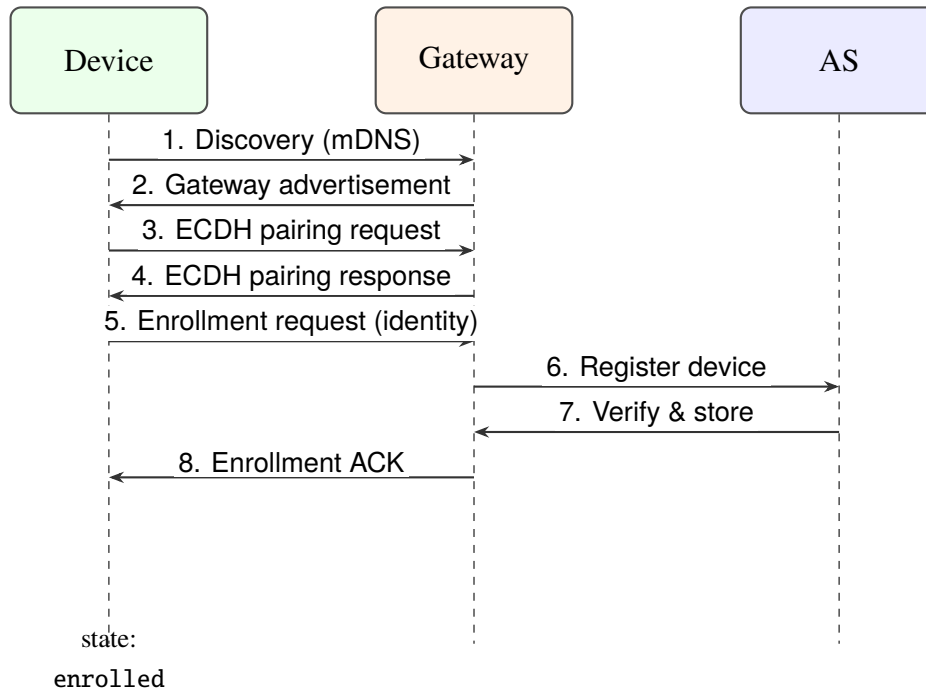


Figure 3.1: Registration flow. The device is discovered, paired via ECDH, and enrolled into the AS registry with its identity material.

At the end of a successful registration, the AS holds the device’s public identity material and the gateway holds a record that the device may subsequently request tokens. The device itself transitions to the **enrolled** state. A failed registration leaves the device in **unregistered** and increments its failure counter at the gateway.

Step	Source	Destination	Event
1	Device	Gateway	discovery
2	Gateway	Device	gateway_advertisement
3	Device	Gateway	pairing_request
4	Gateway	Device	pairing_response
5	Device	Gateway	enrollment_request
6	Gateway	AS	enrollment_request (<i>relayed</i>)
7	AS	Gateway	enrollment_confirmed
8	Gateway	Device	enrollment_confirmed (<i>relayed</i>)

Table 3.1: Registration flow steps. Ending state: **enrolled**.

3.2 Authentication Flow

Authentication is the recurring process by which an enrolled device obtains a scoped, time-limited token and uses it to open a broker session. It consists of five steps: token request, PoP challenge, token issuance, CONNECT to the broker, and successful CONNACK. All five steps traverse the

gateway, which is where the corresponding events are logged.

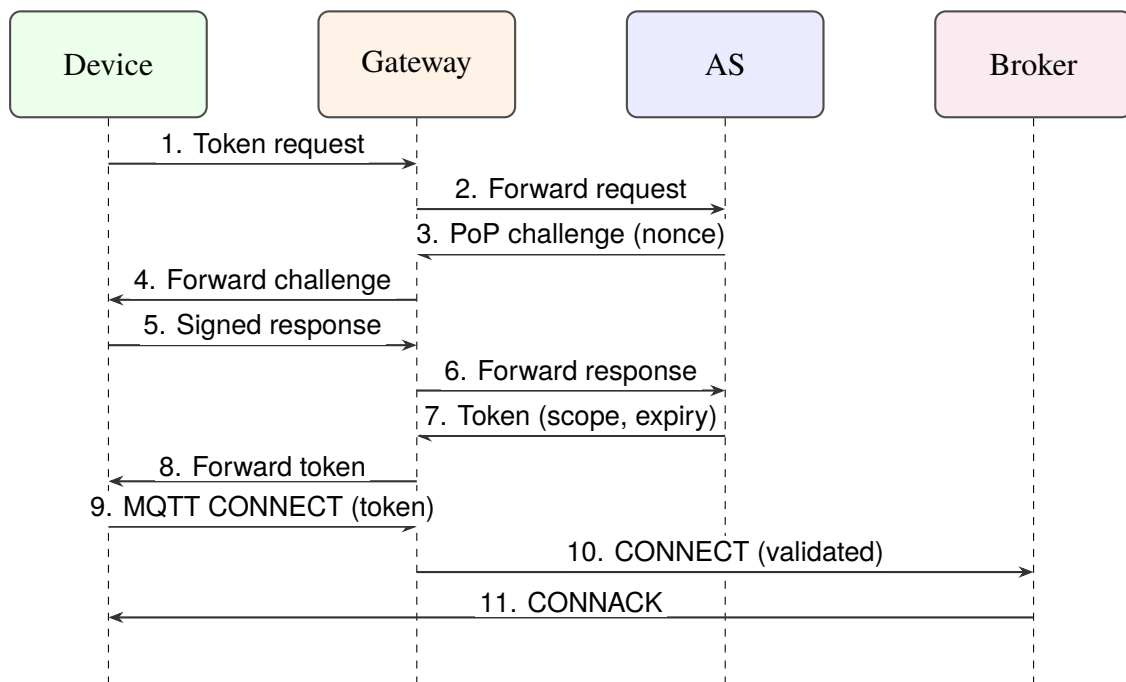


Figure 3.2: Authentication flow. Every message traverses the gateway. The AS issues a scoped token only after a successful PoP challenge; the broker opens a session only after the gateway has validated the token.

Two properties of this flow matter for detection. First, the nonce in the PoP challenge is single-use: a signed response bearing a nonce that has already been consumed within the challenge window is a strong replay signal. Second, the token’s scope is bound to the requesting `device_id`: any subsequent CONNECT that presents this token from a session claiming a different identity is an impersonation signal.

Step	Source	Destination	Event
1	Device	Gateway	authentication_request
2	Gateway	AS	authentication_request (relayed)
3	AS	Gateway	challenge_sent
4	Gateway	Device	challenge_sent (relayed)
5	Device	Gateway	response_sent
6	Gateway	AS	response_sent (relayed)
7	AS	Gateway	token_issued
8	Gateway	Device	token_issued (relayed)
9	Device	Gateway	token_presented
10	Gateway	Broker	token_validated
11	Broker	Device	session_opened

Table 3.2: Authentication flow steps. Ending state: `session_open`. The logged event sequence additionally records `nonce_received` (after `challenge_sent`) and `authentication_success` (after `response_sent`), which the diagram folds into the challenge/response exchange.

3.3 Renewal Flow

Tokens are deliberately short-lived. When a token approaches its expiry, the device re-authenticates without tearing down its MQTT session — a capability native to MQTT v5. The renewal flow is a compressed variant of the authentication flow: the device requests a fresh token, completes a new PoP challenge, and the gateway silently substitutes the new token in the session table.

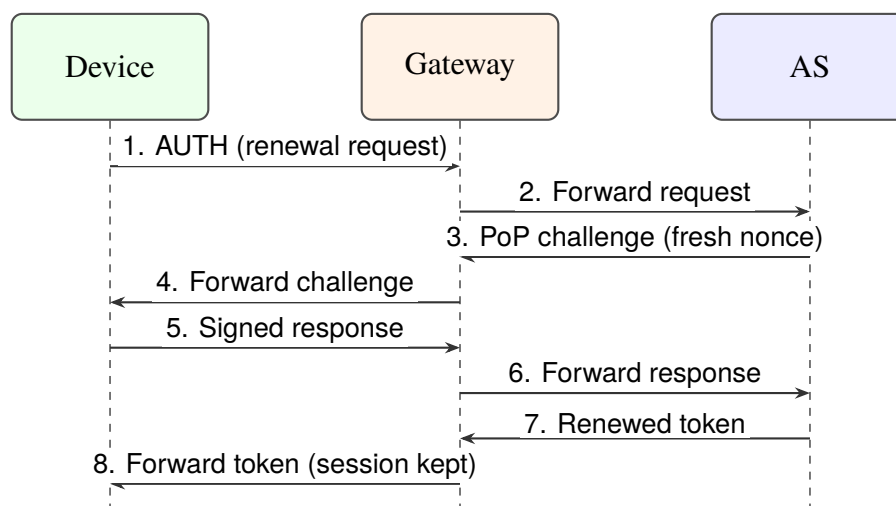


Figure 3.3: Renewal flow. The MQTT session remains open throughout; only the token in the gateway’s session table is replaced.

A legitimate renewal is initiated only when the current token is close to expiry. A renewal request against a token that is still comfortably valid is itself an anomaly signal (*abnormal_renewal*) and is recorded as such.

Step	Source	Destination	Event
1	Device	Gateway	<code>renewal_request</code>
2	Gateway	AS	<code>renewal_request (relayed)</code>
3	AS	Gateway	<code>challenge_sent</code>
4	Gateway	Device	<code>challenge_sent (relayed)</code>
5	Device	Gateway	<code>response_sent</code>
6	Gateway	AS	<code>response_sent (relayed)</code>
7	AS	Gateway	<code>token_issued</code>
8	Gateway	Device	<code>token_issued (relayed)</code>

Table 3.3: Renewal flow steps. The MQTT session stays open throughout; the new token is validated (`token_presented` → `token_validated`) and substituted in the gateway’s session table.

3.4 Attack Flow: Replay

To make the deviation mechanism concrete, Figure 3.4 illustrates a replay attack against the authentication flow. The attacker captures a legitimate signed PoP response from device D and, after the session ends, resubmits it in an attempt to obtain a fresh token in D 's name.

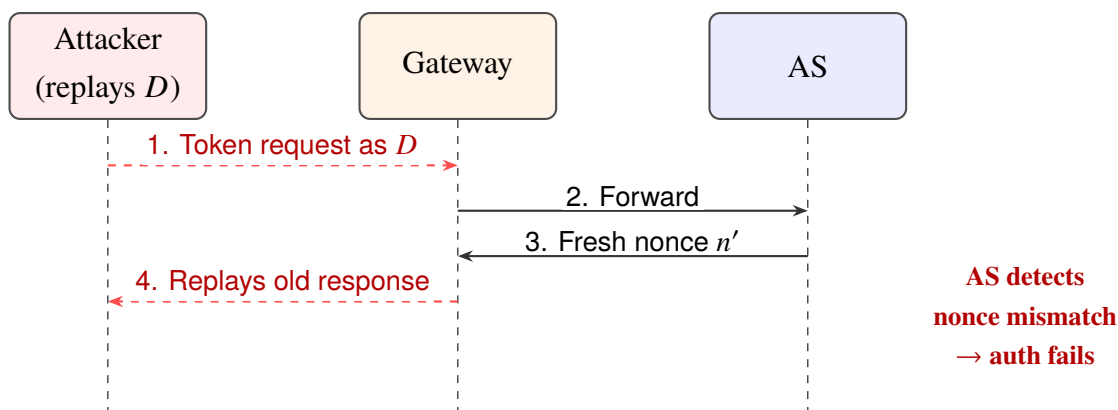


Figure 3.4: Replay attack against the authentication flow. The signed response was valid for a previous nonce; the current challenge carries $n' \neq n$, so the AS rejects the request. The failed attempt is nonetheless recorded by the gateway and labeled `replay_token`.

The important observation is that the attack does not require a new flow — it is the nominal flow with an invalid transition at a specific step. Every attack in Section 4 is modeled the same way: a well-defined deviation from a nominal flow, injected at a specific transition, recorded with the fields that make the deviation observable.

Step	Source	Destination	Event
1	Attacker	Gateway	<code>authentication_request</code>
2	Gateway	AS	<code>authentication_request</code> (<i>relayed</i>)
3	AS	Gateway	<code>challenge_sent</code> (fresh nonce n')
4	Attacker	Gateway	<code>response_sent</code> (replayed, $n \neq n'$) → rejected, labelled <code>replay_token</code>

Table 3.4: Replay attack steps. The injected event deviates from the nominal flow at a single transition; the failed attempt is still recorded and labelled.

Threat Model

The threat model retained for the simulator covers nine authentication-centric attacks. Following the taxonomy in the literature review, two attacker classes can be distinguished:

- **Non-invasive attackers** operate at the network layer. They intercept, replay, or modify traffic but do not hold valid credentials. Their attacks are detectable through protocol-level features: timestamps, nonces, message ordering, token freshness.
- **Invasive attackers** hold valid credentials, typically through prior compromise of a legitimate device or its identity material. Their attacks are indistinguishable from legitimate behavior under static credential checks; they can only be detected by behavioral deviation from a per-device baseline.

The current simulator, however, realizes only a **non-invasive** attacker: every attack in the catalog is injected over the protocol, with no physical access to devices and no assumption of a pre-compromised device, and the generated dataset carries *no* attacker-class field. The two classes above (and the per-attack “class” noted in each subsection and in Table 4.7) are retained only as conceptual background describing how each attack is detectable, not as a property recorded in the data.

4.1 Replay

A replay attack reuses an authentication artifact — a signed challenge response, a token presentation, or a complete authentication sequence — outside the window in which it is valid. The simulator distinguishes three sub-variants: `replay_token` reuses a token past its expiry; `nonce_reuse` reuses a PoP nonce after the challenge that generated it has been consumed; and `duplicate_sequence` replays a complete authentication exchange inside an already-open session. The primary observable signal is the age of the reused artifact at the moment of presentation, captured as `token_age_at_replay` and `nonce_age_at_reuse` in the output records. Non-invasive attacker.

Item	Description	Precondition	Indicator
replay_token	Re-presents a previously issued token past its replay window, with no fresh authentication.	A captured token from an earlier exchange.	token_age_at_replay (large), replay_window_violation=1, registered_device=0
nonce_reuse	Re-sends a PoP nonce after the challenge that produced it was already consumed.	A previously observed challenge nonce.	nonce_age_at_reuse>0, n_nonce_reuses>0
duplicate_sequence	Replays a complete authentication/onboarding sequence inside an already-open session.	An open session plus a captured sequence to replay.	duplicate_session_count=1

Table 4.1: Replay sub-variants: description, precondition, and observable indicator.

4.2 Nonce Reuse and Timestamp Anomalies

Distinct from a full replay, nonce reuse and timestamp inconsistency target the freshness guarantees of the authentication flow specifically. The `timestamp_inconsistency` variant injects an authentication response whose timestamp precedes the challenge that ostensibly prompted it — either because of a clock manipulation on the attacker’s side or because the response was pre-computed. The observable signals are `timestamp_delta_s` (negative or unreasonably small) and, when combined with a repeated nonce, an entry in the gateway’s nonce cache that matches a previously consumed challenge. Non-invasive attacker.

Item	Description	Precondition	Indicator
timestamp inconsistency	Injects an authentication response whose timestamp is shifted backward relative to its challenge (pre-computed or clock-manipulated response).	Ability to set or replay the response timestamp; no valid credential required.	<code>timestamp_delta_s</code> = magnitude of the backward jump (large; 0 for benign sessions)

Table 4.2: Timestamp anomaly variant. The related `nonce_reuse` variant is covered in Table 4.1.

4.3 Impersonation

Impersonation is the presentation of a valid identity by an entity that is not its legitimate holder. In the simulator, this covers two cases. The first, `impersonation`, opens an MQTT session with an identity that the attacker does not actually possess the key for — the PoP challenge fails, and the gateway records the failure. The second, `identity_token_mismatch`, is more subtle: a valid token is presented, but the session opens under a different `device_id` than the one the token was issued to. This is detectable only if the observer compares the token’s bound identity with the session’s claimed identity; the flag `identity_claim_mismatch` in the output records exposes this comparison. Non-invasive attacker.

Item	Description	Precondition	Indicator
<code>impersonation</code>	Opens a session under a claimed identity whose key the attacker does not hold; the token-validation step is bypassed.	A victim <code>device_id</code> to claim (without its credential).	<code>identity_claim_mismatch=1</code> , <code>source_ip_change=1</code> , <code>registered_device=0</code>
<code>identity_token_mismatch</code>	Presents a valid token issued to a <i>different</i> device; the session’s claimed identity differs from the token’s bound identity.	A valid token issued to another device.	<code>token_device_mismatch=1</code> , <code>identity_claim_mismatch=1</code>

Table 4.3: Impersonation variants: description, precondition, and observable indicator.

4.4 Flooding and Protocol Abuse

Flooding attacks aim to exhaust the resources of the gateway or the broker by generating a high rate of authentication or connection requests. The simulator injects three flooding variants: `abnormal_failure_rate` (a burst of failed authentication attempts from a single device, which should trigger local rate-limiting), a CONNECT-flood variant borrowed from the DoS/DDoS-MQTT-IoT dataset (a high rate of MQTT CONNECT packets with invalid credentials), and a delayed-CONNECT variant that completes the TCP handshake but stalls before sending the CONNECT payload, holding gateway resources open. The observable signals are per-device request rates and per-source connection burst counts, both aggregated over sliding windows. Conceptually achievable by either attacker class; injected non-invasively here.

Item	Description	Precondition	Indicator
abnormal_failure_rate	A burst of failed authentication attempts that continues after the device should already be locked out (authentication attempted from the blocked state).	A device that has already crossed the failure threshold (≥ 3 failures $\rightarrow$ blocked).	failed_auth_count, n_failures, n_authentication_failure, failure_rate

Table 4.4: Flooding variant currently implemented. The CONNECT-flood and delayed-CONNECT variants described in the text are not yet injected .

4.5 Access Without Authentication

A device may attempt to publish or subscribe before completing the authentication flow, either as a result of a broken client implementation or as a deliberate probing attempt. The `access_without_auth` variant models this by emitting a PUBLISH or SUBSCRIBE packet while the device's lifecycle phase is still `pairing` or `enrolled` (rather than `authenticated`). The gateway rejects the packet; the rejection, together with the source device's phase at the time of the attempt, is recorded. This is a low-sophistication signal but a frequent one in real traffic, and the simulator emits it to keep the detector honest about baseline behavior. Non-invasive attacker.

Item	Description	Precondition	Indicator
access_without_auth	Emits an access request (PUBLISH/SUBSCRIBE) before authentication completes — while the device is still unregistered, paired, enrolled, or mid-authentication.	An unauthenticated device that attempts resource access.	unauthorized_access_attempt=1, steps_before_access, registered_device=0

Table 4.5: Access-without-authentication variant.

4.6 Abnormal Renewal

A renewal request issued against a token that is still comfortably within its validity window is anomalous. Legitimate renewals are triggered by proximity to expiry; an early renewal suggests either a misconfigured client or an attacker attempting to churn tokens to defeat detection based on token identity. The variant `abnormal_renewal` captures this behavior. The observable signal is the ratio of the token's remaining lifetime to its total lifetime at the moment of the renewal

request: values close to 1 indicate an anomalous renewal. Conceptually an invasive capability (it assumes a compromised device that already holds a valid token); injected non-invasively in the simulator, like the others.

Item	Description	Precondition	Indicator
abnormal_renewal	Requests a token renewal while the current token is still fresh.	A device holding a still-valid, recently issued token.	n_renewal_request>0, re_auth_required

Table 4.6: Abnormal-renewal variant.

4.7 Attack Summary

Table 4.7 consolidates the nine variants, their *conceptual* attacker class, the flow they target, and the primary observable that exposes them.

Variant	Class	Target flow	Primary observable
replay_token	Non-invasive	Auth / Renewal	token_age_at_replay exceeds validity
nonce_reuse	Non-invasive	Auth / Renewal	Nonce present in gateway cache
timestamp_inconsistency	Non-invasive	Auth	timestamp_delta_s ≤ 0 or too small
duplicate_sequence	Non-invasive	Auth	Full sequence replayed inside open session
access_without_auth	Non-invasive	Session	PUBLISH/SUBSCRIBE from non-authenticated phase
abnormal_failure_rate	Both	Auth	Failure count exceeds threshold in window
impersonation	Non-invasive	Auth	PoP verification fails on claimed identity
identity_token_mismatch	Non-invasive	Session	Token device_id $\neq$ session device_id
abnormal_renewal	Invasive	Renewal	Token remaining lifetime close to full

Table 4.7: Threat catalog. Each variant is injected by the scenario script as a well-defined deviation from a nominal flow. The *Class* column reports the attacker capability each attack would conceptually require; in the current simulator all variants are injected non-invasively and no attacker-class field is emitted in the dataset.

Observable Variables

The simulator’s output is a stream of labeled event records. Each record corresponds to a single message observed at the gateway — an authentication step, a token exchange, or a broker interaction — and carries the fields needed to reconstruct the context in which the event occurred. This section defines the fields, grouped by the aspect they describe.

5.1 Session and Identity Fields

These fields locate the event in the scenario and link it to the device it concerns. `scenario_id` identifies the top-level simulation run; `session_id` identifies the specific device-session within it; `device_id` and `gateway_id` identify the two endpoints; `source_ip` records the network origin of the request; `registered_device` is a Boolean indicating whether the device exists in the AS registry at the time of the event; and `credential_status` captures the state of the device’s identity material (`valid`, `expired`, `revoked`).

5.2 State-Machine Transition Fields

These fields describe the transition the message represents. `event_type` is the message name (e.g. `token_request`, `connect`, `publish`); `previous_state` and `new_state` are the device’s lifecycle phases before and after the event; `result` is one of `success`, `fail`, or `timeout`; `failure_reason` carries the specific rejection code when applicable (e.g. `nonce_stale`, `token_expired`, `scope_violation`); and `retry_count` tracks how many times the same transition has been attempted from the same source.

5.3 Token and Nonce Fields

These fields expose the freshness and binding properties of the authentication artifacts. `token_id` is the unique identifier of the token; `token_expiry` is its absolute expiry timestamp; `token_scope` is the set of topics it authorizes; `nonce` is the PoP challenge value; and `identity_claim` records

the identity the presenter claims, which may differ from the identity bound to the token being presented.

5.4 Network and MQTT Fields

These fields describe the transport and application-layer characteristics of the event. `tcp_rtt` is the round-trip time on the underlying TCP connection; `packet_rate` is the current per-source packet rate at the gateway; `qos_level` is the requested MQTT QoS (0, 1, or 2); `requested_topic` is the topic in a PUBLISH or SUBSCRIBE packet; `message_rate` is the current per-device MQTT message rate; `payload_length` is the size in bytes of the MQTT payload; and `connack_code` is the return code the broker issued on the last CONNECT.

5.5 Temporal and Session-Aggregate Fields

These fields capture time-based properties needed to detect replay, flooding, and behavioral drift. `timestamp` is the absolute time of the event; `delay_since_previous_event` is the interval since the previous event on the same session; `session_duration_s` is the elapsed time since session opening; `failure_rate` is the fraction of failed events on the session in a sliding window; `n_state_jumps` counts state transitions on the session; and `n_failures` counts failed events on the session.

5.6 Attack-Variant Signal Fields

These fields are engineered features designed to expose specific attack variants directly. They are computed by the gateway at logging time: `token_age_at_replay` (elapsed time between token issuance and its reuse), `nonce_age_at_reuse` (elapsed time between nonce issuance and its reuse), `timestamp_delta_s` (signed difference between event and challenge timestamps), `identity_claim_mismatch` (Boolean: token identity $\neq$ session identity), `unauthorized_access_attempt` (Boolean: PUBLISH/SUBSCRIBE from non-authenticated phase), and `behavior_deviation_score` (a per-device rolling deviation from message rate, payload size, and topic access baselines).

5.7 Ground-Truth Labels

These fields are the supervision targets for the detection layer. They are known to the simulator because the scenario script injects the anomalies deliberately. `is_anomaly` is a Boolean flag; `attack_type` is one of the nine variants defined in Section 4 (or `none` for legitimate events);

`attack_phase` indicates the lifecycle phase in which the anomaly was injected; and `severity` is a qualitative severity rating.

5.8 Summary Table

Table 5.1 lists the fields grouped by category, with a short indication of the detection role each plays.

Category	Example fields	Role in detection
Session & identity	<code>scenario_id</code> , <code>session_id</code> , <code>device_id</code> , <code>gateway_id</code> , <code>source_ip</code> , <code>registered_device</code> , <code>credential_status</code>	Entity linkage; separates enrolled devices from forged identities
State transitions	<code>event_type</code> , <code>previous_state</code> , <code>new_state</code> , <code>result</code> , <code>failure_reason</code> , <code>retry_count</code>	Invalid transition detection; access-without-auth
Token & nonce	<code>token_id</code> , <code>token_expiry</code> , <code>token_scope</code> , <code>nonce</code> , <code>identity_claim</code>	Replay-window enforcement; nonce-reuse; impersonation
Network & MQTT	<code>tcp_rtt</code> , <code>packet_rate</code> , <code>qos_level</code> , <code>requested_topic</code> , <code>message_rate</code> , <code>payload_length</code> , <code>connack_code</code>	Rate anomalies; unauthorized topic access; flooding
Temporal & aggregates	<code>timestamp</code> , <code>delay_since_previous_event</code> , <code>session_duration_s</code> , <code>failure_rate</code> , <code>n_state_jumps</code> , <code>n_failures</code>	Timing-based replay; abnormal failure rate; duplicate sequence
Attack-variant signals	<code>token_age_at_replay</code> , <code>nonce_age_at_reuse</code> , <code>timestamp_delta_s</code> , <code>identity_claim_mismatch</code> , <code>unauthorized_access_attempt</code> , <code>behavior_deviation_score</code>	Engineered features specific to each attack sub-type
Ground-truth labels	<code>is_anomaly</code> , <code>attack_type</code> , <code>attack_phase</code> , <code>severity</code>	Supervision targets for binary and multi-class classifiers

Table 5.1: Observable variables emitted by the simulator, grouped by category.

5.9 Dataset Structure

Each run writes three complementary, fully-labelled views of the same sessions to `simulator/data/output/` (plus an optional Parquet copy of the feature table):

- **Event log (JSON)** — one object per session, holding the full nested event sequence and the session context.
- **Event log (CSV)** — one row per authentication event (26 columns); the sequence-aware view.
- **Feature table (CSV)** — one row per session (103 columns); the flat, ML-ready view that merges the per-event aggregates with the session-level context.

The feature table is the primary input to the detection pipeline. Its columns are grouped into identifiers, network/pairing, enrollment and authentication, authorization, MQTT session, continuous re-authentication, step latencies, temporal aggregates, per-event-type counts, state coverage, failure and token/identity signals, four label columns, and eight per-anomaly signals. Table A.1 (appendix) lists every column with its type; Table A.2 lists the event-log schema. The four label columns are `is_anomaly` (binary), `attack_type` (normal or one of the nine anomaly labels), `attack_phase` (the event type at which the anomaly was injected), and `severity` (none/medium/high).

Detection Targets

The variables defined in Section 5 are meant to support an AI-based anomaly detection layer that consumes the simulator’s output. Detection is framed as a supervised machine-learning problem: the simulator emits ground-truth labels alongside every session (Section 5), and classifiers are trained on the resulting feature table to recover those labels from the observable variables alone. This section states what the detection layer is expected to distinguish and, for each attack variant, which subset of variables carries the signal — linking every attack in the threat model back to the observable variables that expose it.

Detection operates on the *per-session feature table* (one row per session), which merges the session-level context with aggregates computed over the event sequence. The finer-grained *event log* (one row per event) remains available for sequence-aware models. Unless stated otherwise, the fields named below are columns of the feature table.

6.1 Detection Objectives

The reference detection pipeline trains three supervised ML baselines — Logistic Regression, a Decision Tree, and a Random Forest — and realises two objectives, leaving a third as a documented extension.

Binary detection distinguishes legitimate sessions from anomalous ones — the coarsest task, supported by the `is_anomaly` label.

Multi-class detection distinguishes among the nine attack variants, supported by the `attack_type` label. Both objectives are scored with accuracy, precision, recall, weighted F_1 , ROC-AUC, a confusion matrix, a per-class report, and feature importances, consolidated in a sorted model-comparison table.

Behavioral detection — identifying an attacker whose static credential checks pass but whose behaviour deviates from a per-device baseline — is *not* an objective of the current dataset. Every attack in the threat model is injected non-invasively (Section 4), so no session carries valid credentials with anomalous behaviour, and the dataset provides no per-device baseline. The graded `trust_score` and `behavior_deviation_score` features are emitted as a hook for this

future objective — which would naturally use unsupervised or semi-supervised ML rather than the supervised setup used here — not as a target the present labels can evaluate.

6.2 Attack-to-Signal Mapping

Table 6.1 enumerates each attack variant with the observable variables that primarily expose it. Fields listed as *primary* are the ones a detector should key on directly; fields listed as *contextual* are the ones needed to disambiguate the event from a similar legitimate one.

Attack variant	Primary signal fields	Contextual fields
replay_token	token_age_at_replay, replay_window_violation	registered_device, trust_score
nonce_reuse	n_nonce_reuses, nonce_age_at_reuse	n_events, trust_score
timestamp_inconsistency	timestamp_delta_s	min_delay_s, mean_delay_s
duplicate_sequence	duplicate_session_count, n_state_jumps	n_events, message_rate
access_without_auth	unauthorized_access_attempt, steps_before_access	registered_device, reached_session_open
abnormal_failure_rate	failed_auth_count, n_failures, failure_rate	n_retries, packet_rate
impersonation	identity_claim_mismatch, source_ip_change	registered_device, trust_score
identity_token_mismatch	token_device_mismatch, identity_claim_mismatch	registered_device, claimed_device_id
abnormal_renewal	re_auth_required, n_renewal_request	trust_score, behavior_deviation_score

Table 6.1: Attack-to-signal mapping. Primary fields are the direct evidence of the attack; contextual fields disambiguate it from legitimate behaviour.

Note that `impersonation` and `identity_token_mismatch` both raise `identity_claim_mismatch`; they are separated by `source_ip_change` versus `token_device_mismatch`.

6.3 Label Leakage and Honest Evaluation

Several attacks — `impersonation`, `identity_token_mismatch`, `access_without_auth`, and `abnormal_failure_rate` — terminate the session at the point of injection. Left unmitigated, this makes attack sessions systematically shorter than normal ones, so purely structural features such as `n_events`, `reached_session_open`, or `visited_states` would separate

the classes trivially, and any reported score would reflect that shortcut rather than a genuine authentication signal.

The simulator counters this in three ways. First, a configurable fraction of the in-session replay and renewal anomalies *resume* the legitimate flow after the injected event, so those attack sessions still reach `session_opened`, `access_granted`, and `session_closed`. Second, benign sessions are generated with a broad, both-parity length distribution that includes early drop-outs and overlaps the attack-session lengths. Third, the preprocessing is leakage-aware: it drops the label-alias columns (`attack_type`, `attack_phase`, and `severity` for the binary target; `is_anomaly`, `attack_phase`, and `severity` for the multi-class target) together with high-cardinality identifiers. Even so, residual structural correlation remains: very high scores should be read with this caveat, and the leakage-aware preprocessing should be preferred when benchmarking.

6.4 Expected Detection Difficulty

Under leakage-aware preprocessing, the nine variants are not equally hard. `replay_token`, `nonce_reuse`, and `access_without_auth` are near-deterministic from a single per-variant feature — `token_age_at_replay` or `replay_window_violation`, `n_nonce_reuses` or `nonce_age_at_reuse`, and `unauthorized_access_attempt` respectively — because the primary signal is a hard rule: the artifact is either fresh or it is not, the phase either permits the action or it does not. `abnormal_failure_rate` is straightforward once the failure counts and the elevated request rate are combined. `impersonation` and `identity_token_mismatch` require joining information across fields but remain deterministic Boolean signals (`identity_claim_mismatch`, `token_device_mismatch`) that a properly trained classifier should recover in full. `abnormal_renewal` is the hardest: no dedicated remaining-lifetime ratio is emitted as a feature, and a legitimate renewal session shares the same `re_auth_required` and `n_renewal_request` values, so the two overlap heavily and only the graded `trust_score` and `behavior_deviation_score` carry weak, indirect evidence.

6.5 Non-Objectives

Two categories of detection are explicitly out of scope for the first version of the detector. Detection of *unseen* attack variants — variants the scenario engine does not inject — is not evaluated: classifiers trained on synthetic data are not expected to generalise to attack classes that were never modelled. Detection of *coordinated* attacks across multiple devices (Sybil, distributed DoS, bad-mouthing) is out of scope for the same reason: the threat model in Section 4 injects each attack from a single source. The two flooding variants noted for future work (CONNECT-flood and delayed/half-open CONNECT) and the invasive-attacker track outlined above are the natural next extensions.

Authentication State Machine

Every session is driven through a finite state machine (FSM) that encodes the authentication lifecycle. The machine has **23 states** and **53 valid transitions** over **28 event types**. Normal generation walks only valid transitions; each anomaly is produced by forcing one of **25 invalid** (state, event) pairs (Section 7.3). The transition table is the single source of truth: any (state, event) pair absent from it is, by construction, a candidate anomaly. This is the fine-grained machine that drives event generation; the coarse device-lifecycle view in Section 2.1 (Table 2.1) is an abstraction of it.

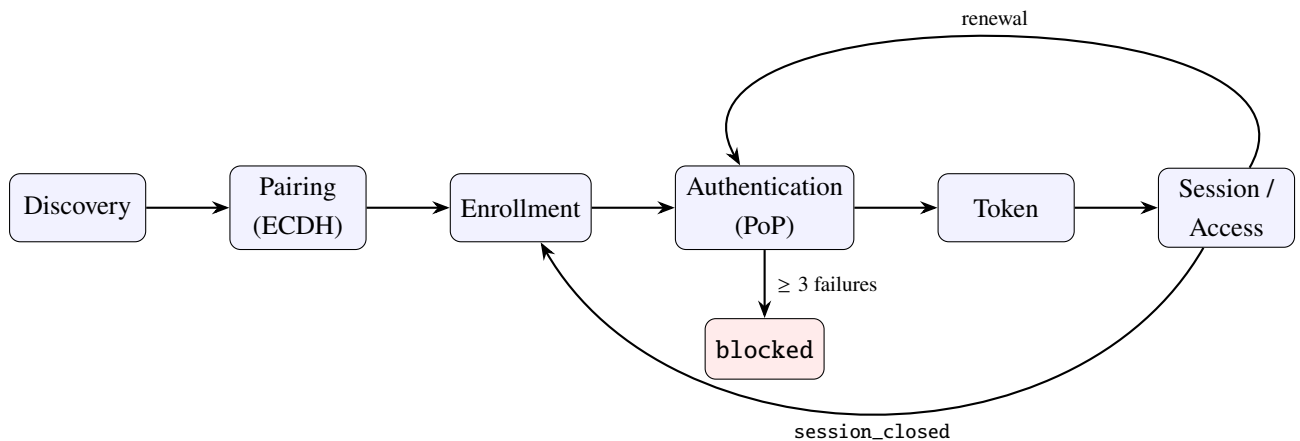


Figure 7.1: Phase-level view of the authentication state machine. Each phase expands into the fine-grained states of Table 7.1; the complete edge set is Table 7.2.

7.1 States

Table 7.1 lists the 23 states grouped by lifecycle phase. **registered** is a backward-compatible alias of **enrolled**; **blocked** is entered automatically after repeated failures (Section 7.3); **revoked** is a reserved guard state not reached by the nominal flow.

State	Meaning
<i>Onboarding</i>	
unregistered	Initial state; device unknown to the AS.
discovered	Gateway located (post-discovery).
pairing	ECDH handshake in progress.
paired	Confidential channel established; identity not yet verified.
enrolling	Identity being verified by the AS.
enrolled	Identity recorded; resting state between sessions.
registered	Legacy alias of enrolled .
<i>Authentication</i>	
auth_requested	Authentication / token request issued.
challenge_issued	PoP challenge sent; awaiting response.
response_sent	Signed challenge response submitted.
authenticated	Authentication succeeded.
auth_failed	Authentication failed.
<i>Token</i>	
token_issued	Token minted by the AS.
token_presented	Token presented for validation.
token_validated	Token validated.
renewal_requested	Mid-session token renewal in progress.
token_expired	Token past its lifetime.
<i>Session & access</i>	
session_open	MQTT session open.
access_requested	Publish/subscribe request pending authorization.
access_granted	Publish/subscribe request authorized.
access_denied	Publish/subscribe request denied (e.g. topic scope).
<i>Guard / terminal</i>	
blocked	Device locked out after repeated failures (rate-limiting).
revoked	Reserved guard state for revoked token/identity.

Table 7.1: The 23 states of the authentication FSM.

7.2 Valid Transitions

Table 7.2 lists all 53 valid transitions, grouped by phase. Each row is a (from state, event) → to state rule. The control events `retry`, `timeout`, and `disconnect` are valid from several states and route back to `auth_requested`, `enrolled`, or `auth_failed`.

Table 7.2: The 53 valid transitions of the authentication FSM.

From state	Event	To state
<i>Discovery</i>		
unregistered	discovery	discovered
discovered	gateway_advertisement	discovered
<i>Pairing</i>		
discovered	pairing_request	pairing
pairing	pairing_response	paired
<i>Enrollment</i>		
paired	enrollment_request	enrolling
enrolling	enrollment_confirmed	enrolled
<i>Registration (legacy alias)</i>		
unregistered	registration_request	enrolled
enrolled	registration_request	enrolled
enrolled	registration_confirmed	enrolled
registered	registration_request	enrolled
registered	registration_confirmed	enrolled
<i>Authentication initiation</i>		
enrolled	authentication_request	auth_requested
registered	authentication_request	auth_requested
token_expired	authentication_request	auth_requested
auth_failed	authentication_request	auth_requested
<i>Challenge-response</i>		
auth_requested	challenge_sent	challenge_issued
renewal_requested	challenge_sent	challenge_issued
challenge_issued	nonce_received	challenge_issued
challenge_issued	response_sent	response_sent
<i>Authentication result</i>		
response_sent	authentication_success	authenticated
response_sent	authentication_failure	auth_failed
<i>Token issuance</i>		

continued on next page

(continued)

From state	Event	To state
authenticated	token_issued	token_issued
renewal_requested	token_issued	token_issued
<i>Token presentation & validation</i>		
token_issued	token_presented	token_presented
token_presented	token_validated	token_validated
token_presented	token_rejected	auth_failed
<i>Session open</i>		
token_validated	session_opened	session_open
<i>Resource access</i>		
session_open	access_request	access_requested
access_granted	access_request	access_requested
access_denied	access_request	access_requested
access_requested	access_granted	access_granted
access_requested	access_denied	access_denied
<i>Token renewal</i>		
session_open	renewal_request	renewal_requested
access_granted	renewal_request	renewal_requested
<i>Token expiry</i>		
token_validated	token_expired	token_expired
session_open	token_expired	token_expired
token_expired	renewal_request	renewal_requested
<i>Session close</i>		
session_open	session_closed	enrolled
access_granted	session_closed	enrolled
access_denied	session_closed	enrolled
token_validated	session_closed	enrolled
<i>Retry</i>		
auth_failed	retry	auth_requested
token_expired	retry	auth_requested
access_denied	retry	access_requested

continued on next page

(continued)

From state	Event	To state
<i>Timeout</i>		
auth_requested	timeout	enrolled
challenge_issued	timeout	enrolled
response_sent	timeout	enrolled
token_presented	timeout	auth_failed
session_open	timeout	enrolled
<i>Disconnect</i>		
session_open	disconnect	enrolled
access_granted	disconnect	enrolled
authenticated	disconnect	enrolled
token_validated	disconnect	enrolled

7.3 Invalid Transitions (Anomaly Injection)

An anomaly is generated by forcing the machine into a chosen predecessor state and firing an event that is *absent* from Table 7.2, raising a transition error that is caught and labelled. Table 7.3 maps each of the nine anomaly labels to the invalid (state, event) pairs it may use; one pair is drawn at random per attack session. Separately, three consecutive `auth_failed` outcomes move the machine to `blocked` automatically (gateway rate-limiting); the `abnormal_failure_rate` anomaly is precisely an authentication attempt fired from that `blocked` state.

Anomaly	From state	Invalid event
replay_token	enrolled	token_presented
	session_open	token_presented
	auth_failed	token_presented
nonce_reuse	enrolled	nonce_received
	authenticated	nonce_received
	session_open	nonce_received
	token_validated	nonce_received
timestamp_inconsistency	enrolled	response_sent
	auth_requested	response_sent
access_without_auth	unregistered	access_request
	paired	access_request
	enrolled	access_request
	auth_requested	access_request
	response_sent	access_request
abnormal_failure_rate	blocked	authentication_request
	blocked	discovery
impersonation	auth_failed	session_opened
	token_presented	session_opened
identity_token_mismatch	enrolled	session_opened
	authenticated	session_opened
abnormal_renewal	token_issued	renewal_request
	token_validated	renewal_request
	token_presented	renewal_request
duplicate_sequence	session_open	discovery
	authenticated	discovery

Table 7.3: The 25 invalid (state, event) pairs, grouped by the nine anomaly labels. Each pair is deliberately absent from Table 7.2; firing it raises the transition error the event engine catches and labels.

Dataset Schema

Table A.1: Feature table schema (one row per session, 103 columns). Types are those of the generated CSV: *binary* denotes an integer $\in \{0, 1\}$; *categorical* denotes a string over a small fixed value set.

Column	Type	Description
Identifiers & identity		
scenario_id	string	Unique id of the generated scenario.
session_id	string	Unique id of the session.
device_id	string	Real device UUID.
gateway_id	string	Id of the mediating gateway (one of the pool).
claimed_device_id	string	Identity claimed on the wire (differs from device_id under impersonation).
source_ip	string	Device source IP.
registered_device	binary	1 if the presented credential is genuine, 0 if forged/stolen.
source_connection_count	integer	Connections attributed to the source.
battery_level	float	Reported battery level (%).
Network & pairing		
tcp_flags	categorical	Observed TCP flag combination.
connection_duration	float	TCP connection duration (ms).
tcp_rtt	float	TCP round-trip time (ms).
packet_rate	float	Packets per second.
inter_arrival_time	float	Mean inter-packet time (ms).
frame_length	integer	Frame length (bytes).
tcp_segment_len	integer	TCP segment length (bytes).
pairing_result	binary	1 if the ECDH pairing succeeded.
pairing_latency_ms	float	Pairing (ECDH) latency (ms).

continued on next page

(continued)

Column	Type	Description
Enrollment & authentication		
mqtt_msg_type	categorical	MQTT message type for the session.
connect_flags	integer	MQTT CONNECT flags byte.
clean_session	binary	MQTT clean-session flag.
username_present	binary	Whether a username was present.
password_length	integer	Credential length (bytes).
keep_alive	integer	MQTT keep-alive (s).
mqtt_version	integer	MQTT protocol version (constant: 5).
connack_code	categorical	CONNACK / enrollment result code.
auth_result	binary	1 if authentication succeeded.
auth_latency_ms	float	Authentication latency (ms).
failed_auth_count	integer	Failed auth attempts in the session.
Authorization		
requested_topic	string	MQTT topic requested.
topic_length	integer	Length of the requested topic.
operation	categorical	Requested operation (publish / pub_sub).
authorization_result	binary	1 if the pub/sub request was authorized.
topic_scope_violation	binary	1 if the topic fell outside the allowed scope.
retain_flag	binary	MQTT retain flag.
MQTT session		
message_id	integer	MQTT message identifier.
duplicate_flag	binary	MQTT DUP flag.
payload_length	integer	Payload size (bytes).
payload_hash	string	Blake2s hash of a payload sample.
qos_level	integer	Negotiated QoS (0/1/2).
message_rate	float	Messages per second.
byte_rate	float	Bytes per second.
Continuous re-authentication / Zero Trust		
trust_score	float	Continuous trust score in [0, 1].
re_auth_required	binary	1 if re-authentication was triggered.
session_present	binary	1 if an MQTT session was opened.
source_ip_change	binary	1 if the source IP changed within the session.

continued on next page

(continued)

Column	Type	Description
replay_window_violation	binary	1 if a token was presented outside the replay window.
Step latencies		
s1_latency_ms– s6_latency_ms	float	Per-phase latency (ms): discovery/registration, pairing/challenge, auth, token issuance, session open, re-auth (6 columns).
Temporal aggregates		
session_duration_s	float	Session span (s).
mean_delay_s	float	Mean inter-event delay (s).
max_delay_s	float	Maximum inter-event delay (s).
min_delay_s	float	Minimum non-zero inter-event delay (s).
Per-event-type counts		
n_events	integer	Number of events in the session.
n_registration_request– n_disconnect	integer	Count of each event type; one column per type (22 columns).
State coverage		
visited_states	integer	Number of distinct states visited.
reached_session_open	binary	1 if SESSION_OPEN was reached.
reached_access_granted	binary	1 if ACCESS_GRANTED was reached.
reached_authenticated	binary	1 if AUTHENTICATED was reached.
Failure signals		
n_failures	integer	Number of failure-result events.
n_retries	integer	Maximum retry count observed.
failure_rate	float	Failures divided by total events.
Token / identity signals		
n_token_reuses	integer	Repeated token_id occurrences.
n_nonce_reuses	integer	Repeated nonce occurrences.
identity_mismatch	binary	1 if any event's claimed id ≠ device_id.
n_state_jumps	integer	Number of non-contiguous state transitions.
Labels		
is_anomaly	binary	1 if the session is an attack.

continued on next page

(continued)

Column	Type	Description
attack_type	categorical	normal or one of the nine anomaly labels.
attack_phase	categorical	Event type at which the anomaly was injected (none if normal).
severity	categorical	none / medium / high.
Per-anomaly signals		
token_age_at_replay	float	Age (s) of the replayed token (replay_token).
nonce_age_at_reuse	float	Age (s) of the reused nonce (nonce_reuse).
timestamp_delta_s	float	Magnitude (s) of the backward timestamp jump (timestamp_inconsistency).
duplicate_session_count	integer	Number of replayed sequences (duplicate_sequence).
identity_claim_mismatch	binary	1 if claimed id $\neq$ bound id (impersonation / mismatch).
token_device_mismatch	binary	1 if the token is bound to another device (identity_token_mismatch).
unauthorized_access_attempt	binary	1 if access was attempted before auth (access_without_auth).
steps_before_access	integer	Auth steps completed before the unauthorized access.

Column	Type	Description
event_id	string	Unique event UUID.
event_type	categorical	Lifecycle event type (e.g. token_issued).
timestamp	datetime	Event time, rendered UTC YYYY-MM-DD HH:MM:SS.
delay_since_previous_event	float	Seconds since the previous event in the session.
scenario_id	string	Scenario id (shared across the session).
session_id	string	Session id.
device_id	string	Real device UUID.
gateway_id	string	Mediating gateway id.
auth_server_id	string	Authorization server id (constant).
broker_id	string	MQTT broker id (constant).
previous_state	categorical	State before the event.
new_state	categorical	State after the event.
result	categorical	success / failure / pending.
failure_reason	string	Failure reason, else empty.
retry_count	integer	Retry counter at this event.
token_id	string	Token id (empty for non-token events).
token_expiry	datetime	Token expiry, rendered UTC (empty if none).
token_scope	categorical	Token scope (empty if none).
nonce	string	PoP nonce (empty for non-challenge events).
identity_claim	string	Claimed identity (set under impersonation).
topic	string	MQTT topic / resource (empty if N/A).
resource_id	string	Target resource id (empty if N/A).
firmware_version	categorical	Reported firmware version.
source_context	categorical	normal or attack.
anomaly_label	categorical	Anomaly label (empty for normal events).
attack_type	categorical	Session-level attack label, copied onto each row.

Table A.2: Event-log schema (one row per authentication event, 26 columns).